

CORSO DI LAUREA IN INFORMATICA



Link alla repository

Autori

Michele Antonio Annunziata

Raffaele Coppola

Luigi Auriemma

Domenico Cirillo

Docente

Prof. Fabio Palomba

Indice

1	Introduzione al problema	3
1.1	L'AI nel settore automobilistico	3
1.1.1	Le sfide principali	3
1.2	Il problema affrontato	3
1.2.1	Le origini del progetto	3
1.2.2	Perché Carval?	4
1.2.3	Regressione e classificazione	4
1.3	Specifiche PEAS	4
1.3.1	Caratteristiche dell'ambiente	5
1.4	Struttura della repository	5
2	Trattamento dei dati	6
2.1	Data understanding	6
2.1.1	Raccolta dei dati	6
2.1.2	Documentazione dei dati	7
2.1.3	Esplorazione dei dati	7
2.2	Data preparation	12
2.2.1	Pulizia dei dati	12
2.2.2	Il problema delle scale	15
2.2.3	Ingegneria delle caratteristiche	16
2.2.4	Bilanciamento delle classi	17
3	Sviluppo del modello	19
3.1	Selezione del modello	19
3.1.1	Fase di addestramento	20
3.2	Tecniche di valutazione	22
3.2.1	Metriche di validazione	23
4	Usabilità	25
4.1	Leggi di Lehman	25
4.2	Interfaccia grafica	25

4.2.1	Perché Streamlit?	26
5	Considerazioni finali	27
5.1	Possibili miglioramenti	27
5.1.1	Nei dati	27
5.1.2	Nel modello	27

Elenco delle figure

1	Matrice di correlazione delle caratteristiche numeriche	8
2	Studio delle frequenze delle caratteristiche	9
4	Studio dei valori nulli	14
5	Bilanciamento delle classi	17
6	Validation loss e accuracy al variare di max_depth	19
7	Importanza delle caratteristiche	22
8	Validazione incrociata	23
9	Matrice delle confusioni (primo e ultimo fold)	23
10	Interfaccia grafica	26

Elenco delle tabelle

1	Statistiche descrittive delle caratteristiche numeriche	8
2	Studio dei domini delle caratteristiche	13
3	Valori metriche di validazione (classificazione)	24
4	Valori metriche di validazione (regressione)	24

1 Introduzione al problema

1.1 L'AI nel settore automobilistico

L'intelligenza artificiale e il Machine Learning stanno rivoluzionando numerosi settori, compreso quello della valutazione automobilistica. L'analisi automatizzata dei veicoli consente di ridurre notevolmente l'incertezza e di migliorare i processi decisionali per venditori, acquirenti e istituzioni finanziarie.

1.1.1 Le sfide principali

Come qualsiasi approccio basato sull'AI, l'implementazione di un modello predittivo per la valutazione delle auto presenta alcune criticità fondamentali:

- **qualità e disponibilità dei dati.** La qualità dei dati ha un forte impatto rispetto alle prestazioni del modello.
- **explainability.** Risulta essenziale l'interpretabilità del modello in modo da poter spiegare agli utenti finali come ogni caratteristica influenzi la valutazione finale.

1.2 Il problema affrontato

L'obiettivo principale del progetto è sviluppare un modello di machine learning che esegue un task di apprendimento supervisionato per cui viene predetto il prezzo di un'auto usata sulla base di un'esperienza maturata da un dataset che include diverse caratteristiche delle vetture automobilistiche.

1.2.1 Le origini del progetto

Il progetto Carval nasce da una situazione reale: durante un'uscita di gruppo, l'auto di uno degli autori del progetto si è guastata irreparabilmente. Di conseguenza, la necessità di acquistare un'auto usata ha evidenziato le difficoltà nel determinare il giusto valore di mercato dei veicoli di seconda mano.

La ricerca di una nuova auto ha rivelato diversi problemi, tra cui discrepanze significative tra valutazioni di concessionari e privati e una forte variabilità dei prezzi tra annunci simili.

Da questa esperienza personale nasce Carval: un sistema basato su Machine Learning per analizzare e stimare il valore delle auto usate. Il progetto unisce l'utilità pratica ad un tocco di ironia sulla situazione che l'ha generato, trasformando un inconveniente in un'opportunità per sviluppare uno strumento di valutazione oggettivo per il mercato dell'usato.

1.2.2 Perché Carval?

Carval nasce con l'obiettivo di offrire una valutazione data-driven, aiutando diversi tipi di utenti a prendere decisioni più informate.

Il sistema è stato progettato per rispondere alle esigenze di diversi attori:

- **acquirenti** di auto usate. Chi desidera acquistare un veicolo può utilizzare Carval per verificare se il prezzo richiesto è in linea.
- Venditori **privati** e concessionari. Chi vende un'auto può ottenere una stima realistica del valore del proprio veicolo, facilitando la definizione di un prezzo competitivo.
- Compagnie di **assicurazione**. Le assicurazioni possono impiegare il sistema per stimare il valore residuo delle auto e pianificare meglio le operazioni di vendita o sostituzione.

L'idea alla base di Carval è quella di democratizzare l'accesso a valutazioni affidabili, riducendo l'asimmetria informativa tra venditori e acquirenti.

1.2.3 Regressione e classificazione

Per una maggiore modellazione del problema e sperimentazione delle analisi, si è deciso di affrontarlo da due diverse prospettive: sia come task di regressione, sia come task di classificazione.

Nel caso della regressione, l'obiettivo del modello è predire il prezzo esatto di un veicolo automobilistico, mentre nel caso della classificazione il prezzo è suddiviso in fasce di valore (meno di $10k$, $10k - 20k$ ecc.), ed il modello, anziché stimarne il valore esatto, deve imparare a classificare un'auto all'interno di una determinata fascia di prezzo.

1.3 Specifiche PEAS

Performance. La misura di prestazione adottata prevede l'accuratezza delle stime, valutata tramite metriche standard come l'errore quadratico medio o l'errore medio assoluto per la regressione, mentre per la classificazione si considerano le metriche derivanti dalla matrice di confusione. Inoltre, è fondamentale la capacità del modello di generalizzare su nuovi dati, evitando fenomeni di overfitting.

Environment. L'ambiente in cui opera l'agente tiene conto delle auto rappresentate mediante una serie di caratteristiche.

Actuators. L'output dell'agente è una stima del valore di un'auto sulla base dei dati presi in ingresso.

Sensors. L'agente riceve in input i dati dell'auto immesse dall'utente mediante interfaccia.

1.3.1 Caratteristiche dell'ambiente

Parzialmente osservabile. La completezza dello stato osservabile risulta limitata in quanto il modello non dispone di tutte le informazioni possibili riguardo ad una specifica auto o al mercato in ogni momento. Ad esempio, potrebbero mancare dettagli nascosti come la storia degli incidenti, lo stato effettivo dei componenti meccanici o variazioni imminenti del mercato.

Singolo agente. Il sistema è progettato attorno a un unico agente (regressore/classificatore) che fornisce una stima del valore di un'auto. Non è prevista competizione e/o collaborazione.

Deterministico. Stessi input producono lo stesso output.

Episodico. Ogni previsione sul valore di un'auto è considerata un episodio indipendente. La stima per una determinata auto non influenza direttamente le stime di altre auto.

Statico. Durante il processo di previsione, l'ambiente è statico poiché le caratteristiche dell'auto rilevate in input non cambiano durante il calcolo della stima.

Continuo. Alcune delle variabili in input (come chilometraggio) sono di tipo numerico continue.

1.4 Struttura della repository

La repository è organizzata in modo da garantire una chiara separazione logica tra dati, codice e documentazione. Di seguito viene descritta la struttura delle principali cartelle e file presenti:

- *dataset*: contiene il dataset di partenza reperito da Kaggle e il dataset pre-elaborato.
- *diagrams*: contiene tutti i grafici generati durante lo sviluppo della pipeline;
- *docs*: raccoglie la documentazione e la presentazione relativa al progetto;
- *scripts*: contiene tutti gli script sviluppati per il progetto (*.py* e *.ipynb*).
- *README.md*: fornisce informazioni su come installare, configurare ed utilizzare il progetto;

- *requirements.txt*: contiene la lista delle dipendenze necessarie per eseguire gli script della repository, facilitando la configurazione dell'ambiente di sviluppo.

Questa organizzazione consente una gestione efficiente del progetto, facilitando la manutenibilità e l'eventuale estensione delle funzionalità. La repository è disponibile qui.

2 Trattamento dei dati

2.1 Data understanding

2.1.1 Raccolta dei dati

Per addestrare il modello di Machine Learning, il dataset di riferimento è stato acquisito dalla piattaforma Kaggle, al seguente link: <https://www.kaggle.com/datasets/tunguz/used-car-auction-prices>.

La scelta di utilizzare un dataset di Kaggle, anziché raccogliere dati autonomamente tramite web scraping o con altre tecniche, è motivata da fattori relativi alla **facilità di accesso**, in quanto l'uso di un dataset pubblico consente di garantire la riproducibilità degli esperimenti, evitando problemi legati alla legalità della raccolta dati e alla gestione di grandi volumi di informazioni eterogenee; inoltre, la raccolta di dati tramite scraping potrebbe essere un processo lungo e complesso, con rischi legati all'obsolescenza dei dati e alla necessità di una manutenzione costante degli script di raccolta.

Tuttavia, la reperibilità dei dati rimane una delle sfide principali nell'applicazione dell'IA. I dataset pubblici possono presentare limitazioni in termini di rappresentatività rispetto al mercato reale.

Limiti dei dati. Nonostante l'immediatezza della reperibilità del dataset, esso presenta alcune limitazioni che devono per forza di cose essere tenute in considerazione. Le principali restrizioni riguardano la copertura temporale e la rappresentatività geografica. Infatti, il dataset è composto esclusivamente da dati del mercato nord-americano fino al 2015. Ciò significa che il modello è stato addestrato su informazioni storiche e potrebbe non catturare le tendenze di mercato più recenti, non rendendo del tutto valide le stime ottenute per altri paesi in cui sono presenti dinamiche di mercato differenti.

Tuttavia, uno dei principali ostacoli nell'applicazione dell'intelligenza artificiale alla valutazione delle automobili è l'impossibilità di rappresentare completamente l'intero panorama mondiale dei veicoli disponibili in un singolo dataset. Ogni mercato automobilistico è verosimilmente influenzato da fattori economici, culturali, normativi e tecnologici che variano significativamente da paese a paese.

2.1.2 Documentazione dei dati

Il dataset di partenza è costituito da 558811 osservazioni caratterizzate da 16 feature.

Di seguito è riportata la lista delle caratteristiche con relativa descrizione.

- **year**: anno di produzione dell'auto.
- **make**: marca del veicolo.
- **model**: modello specifico dell'auto.
- **trim**: allestimento o versione del modello.
- **body**: tipo di carrozzeria del veicolo.
- **transmission**: tipo di trasmissione.
- **vin**: codice univoco per identificare il veicolo.
- **state**: stato di registrazione del veicolo.
- **seller**: venditore del veicolo.
- **condition**: condizione generale del veicolo.
- **odometer**: chilometraggio dell'auto.
- **color**: colore carrozzeria auto.
- **interior**: colore degli interni dell'auto.
- **mmr**: indicatore del valore di mercato su base storica.
- **sellingprice**: prezzo effettivo di vendita del veicolo (**target**).
- **saledate**: data in cui l'auto è stata venduta.

2.1.3 Esplorazione dei dati

In Tabella 1 sono riportate parametri statistici standard riguardo le caratteristiche numeriche.

	Count	Mean	Std	Min	Max
year	558,811	2010.04	3.97	1982	2015
condition	547,017	3.42	0.95	1.0	5.0
odometer	558,717	68,323.20	53,397.75	1.0	999,999
mmr	558,811	13,769.32	9,679.87	25.0	182,000
sellingprice	558,811	13,611.26	9,749.66	1.0	230,000

Tabella 1: Statistiche descrittive delle caratteristiche numeriche

Correlazione con il target. Nell'analisi dei dati, la matrice di correlazione è uno strumento essenziale per comprendere le relazioni tra le variabili del dataset. La correlazione misura il grado di associazione lineare tra due variabili e fornisce informazioni cruciali sulla loro dipendenza reciproca.

Il valore della correlazione varia tra -1 e 1 :

- *Correlazione positiva* ($+1$). Quando una variabile aumenta, anche l'altra tende ad aumentare.
- *Correlazione negativa* (-1). Quando una variabile aumenta, l'altra tende a diminuire.
- *Correlazione nulla* (0): Le due variabili non presentano alcuna relazione lineare.

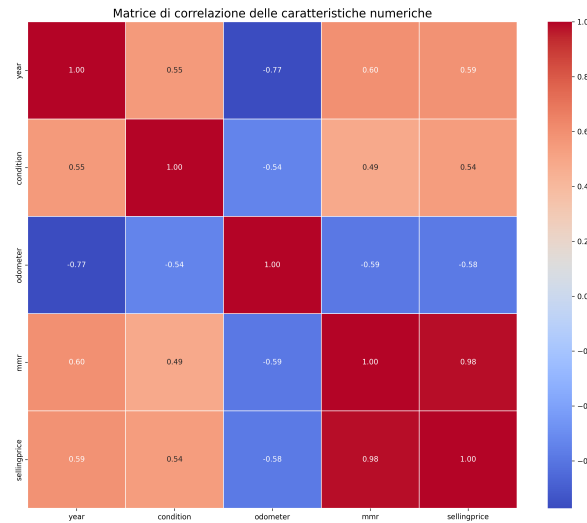


Figura 1: Matrice di correlazione delle caratteristiche numeriche

Dall'analisi della matrice di correlazione delle caratteristiche numeriche presente in Figura 1, si osserva una forte correlazione tra la variabile mmr e sellingprice. Successivamente è spiegata la problematica di questa forte correlazione.

Il problema di data leakage. Un aspetto critico nella costruzione di un modello di machine learning è il rischio di **data leakage**. Tale problema si presenta quando un modello è capace di lavorare in maniera performante ed accurata in fase di addestramento, ma non in fase di rilascio. Ciò è possibile perché magari si utilizzano caratteristiche disponibili nel training ma che non saranno disponibili a "run time" (**leaky predictor**).

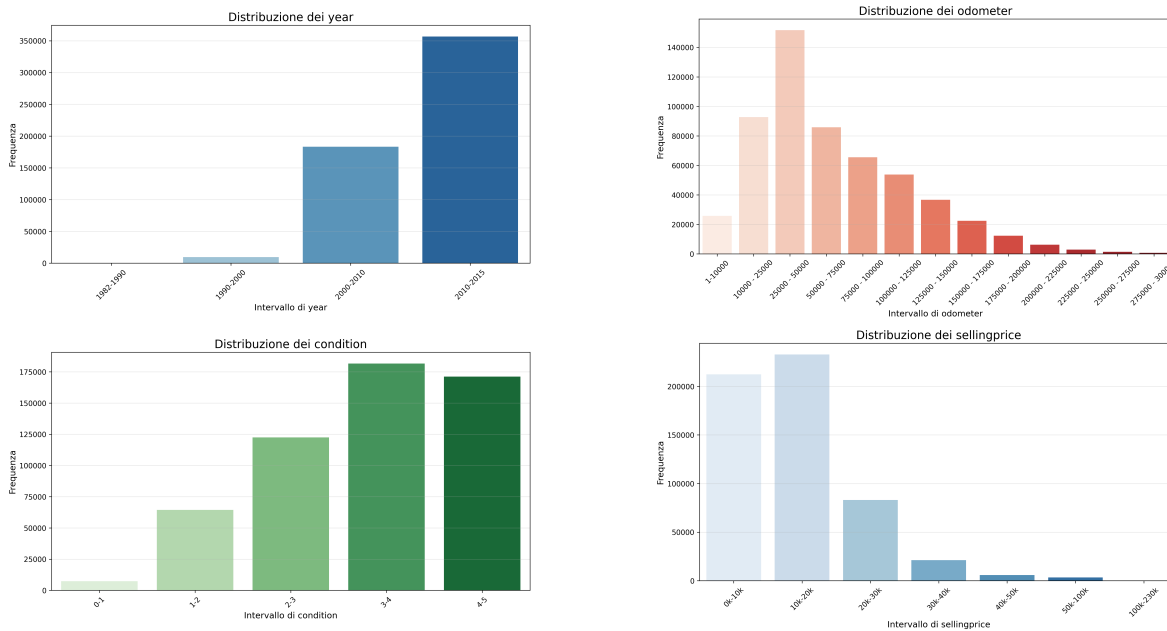


Figura 2: Studio delle frequenze delle caratteristiche

Nel dataset di riferimento, l'uso di mmr come variabile predittiva introduce un rischio significativo di data leakage, in quanto contiene per sua natura informazioni derivate direttamente dal target. Se il valore mmr è stato calcolato utilizzando informazioni disponibili solo dopo la vendita del veicolo, allora il modello potrebbe imparare una relazione artefatta, sfruttando dati che in una situazione reale non sarebbero disponibili al momento della valutazione del prezzo.

Per ovviare al problema, la caratteristica mmr è stata sin da subito esclusa, così da garantire che il modello possa apprendere solamente da caratteristiche disponibili prima della vendita del veicolo.

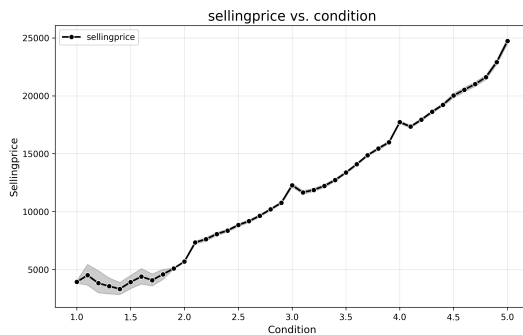
Distribuzione caratteristiche numeriche. In Figura 2 sono riportati i grafici relativi alle frequenze delle caratteristiche numeriche.

Studio variazione target. L'analisi della variazione del target, ovvero il prezzo di vendita (selling-price) in funzione di diverse variabili indipendenti, è una fase cruciale per comprendere le dinamiche di

mercato e identificare eventuali pattern che influenzano il valore delle automobili. Per questo motivo, si è deciso di analizzare alcuni grafici che illustrano l'andamento del prezzo in relazione a caratteristiche chiave dei veicoli.

Da Figura 3a si può notare che all'aumentare del punteggio assegnato alle condizioni aumenta anche il valore del prezzo.

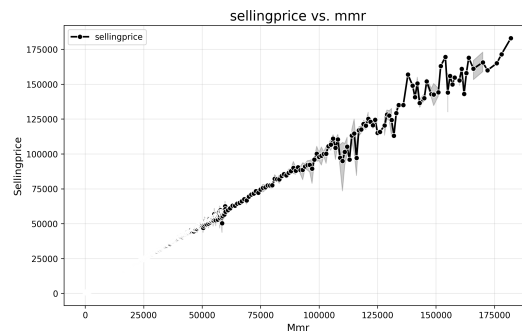
Da Figura 3b si evince che dall'anno 1995 c'è stato un andamento crescente con variazioni positive della pendenza della retta che descrive la relazione.



(a) Condizione contro prezzo



(b) Anno contro prezzo



(c) MMR contro prezzo

La Figura 3c mostra una forte correlazione tra le variabili disaminate: si nota che la retta descrittiva della relazione tra il Manheim Market Report (mmr) e il prezzo di vendita (sellingprice) ha un andamento molto simile alla bisettrice del primo e terzo quadrante del piano cartesiano. Matematicamente, ciò indica che i valori del MMR e del prezzo di vendita sono quasi coincidenti nella maggior parte dei casi, suggerendo una correlazione fortemente lineare (come già evinto dalla Figura 1). Da tali assunzioni, si evidenziano le problematiche discusse nelle sezioni precedenti circa il data leakage.

Pre-elaborazione dei dati. Nel caso specifico di Carval, si sono riscontrate diverse problematiche circa l'ambiguità dei dati di partenza. Un esempio concreto di "rumore" si trova nella variabilità delle

denominazioni relative ad uno stesso marchio di auto. Ad esempio, nel dataset di partenza è presente la stessa marca registrata in diverse forme: 'mercedes', 'mercedes-b', 'mercedes-benz'. Queste variazioni, pur facendo riferimento alla stessa marca, potrebbero essere trattate come voci distinte se non correttamente gestite, causando confusione nelle analisi e introducendo inconsistenza nei dati. Per affrontare questo problema, si è adottata una tecnica di **uniformazione** delle denominazioni. In pratica, tutte le varianti di denominazione vengono consolidate in un'unica forma standard, definita attraverso un banale processo di mapping.

Un ulteriore tipo di rumore è stato riscontrato nelle feature *modello* e *allestimento*. In particolare, è emerso che alcuni campioni presentassero un valore di *allestimento* identico a quello di *modello*. Per ovviare a questa anomalia, si è deciso di sostituire, in tali osservazioni, il valore di *allestimento* con la stringa 'base', garantendo così una maggiore coerenza e uniformità nei dati.

Infine, è stato necessario gestire alcune ambiguità nei dati relativi alle colorazioni dei veicoli. Le variabili *color* e *interior* presentano in alcuni casi valori poco informativi e utili per l'analisi, come il simbolo "-". Per tali osservazioni si è deciso di impostarle come informazioni mancanti e gestirle poi nelle fasi successive della pipeline.

Raggruppamento degli allestimenti. Nel dataset di riferimento, la variabile *trim* rappresenta una versione specifica di un modello di veicolo, spesso indicante allestimenti, dotazioni aggiuntive o pacchetti opzionali. Questa informazione può essere utile nell'analisi predittiva, ma la sua elevata granularità e variabilità lessicale rendono difficile la sua integrazione diretta nei modelli di machine learning: in poche parole, il problema di fondo è che ci sono troppi valori diversi che magari indicano pressoché la stessa versione.

Per affrontare questo problema, si è scelto di raggruppare le diverse varianti in **categorie** semantiche più ampie: *base*, *special edition*, *sport*, *touring*, *luxury* e *other* (tutti i casi che non rientrano in nessuna delle categorie precedenti o sono valori mancanti/ambigui). L'uso di categorie più ampie evita problemi per alcune varianti poco frequenti e garantisce una maggiore interpretabilità.

Raggruppamento delle carrozzerie. Un'operazione simile a quella descritta precedentemente è stata realizzata per la caratteristica *body*. In particolare si è deciso di raggruppare i dati nelle seguenti categorie: *sedan*, *suv*, *hatchback*, *van*, *coupé*, *cabriolet*, *station wagon*, *pickup* e *other*.

Informazioni irrilevanti. Durante il processo di pre-elaborazione del dataset, sono state rimosse alcune variabili che, sebbene presenti nel dataset originale, sono risultate semanticamente irrilevanti per lo scopo predittivo e potrebbero addirittura introdurre un certo tasso di distorsione nei risultati.

Tali analisi, hanno causato la rimozione delle seguenti caratteristiche:

- *mmr*, per le motivazioni discusse nelle sezioni precedenti.
- *vin*, in quanto, trattandosi di un codice univoco, non consente al modello di stabilire dei pattern.
- *seller*, informazione ritenuta futile agli scopi della predizione.
- *saledate*, poiché si è deciso di non trattare esplicitamente dati di tipo temporali.
- *state*, perché i risultati finali dovranno trascendere i confini geografici di registrazione.

Ridenominazione caratteristiche. Per migliorare la comprensione nell'analisi dei dati, è stata effettuata una traduzione delle variabili da inglese ad italiano.

2.2 Data preparation

2.2.1 Pulizia dei dati

Nel contesto di machine learning, dati di bassa qualità possono portare a risultati fuorvianti e ad una generalizzazione scarsa, compromettendo le capacità del modello di apprendere le reali relazioni presenti tra le caratteristiche che si hanno a disposizione. Pertanto, un accurato processo di data cleaning risulta essere di vitale importanza.

Per pulizia dei dati si intende rendere i dati utilizzabili da un modello di ML ed in particolare capire in che modo sopravvivere alla mancanza o al rumore dei dati di partenza.

Indipendentemente dalle altre fasi di pulizia dei dati, è stato applicato un filtro minimo sul prezzo per cui sono stati considerati esclusivamente i campioni con un valore di prezzo maggiore o uguale a 500. Quest'operazione è stata eseguita per rimuovere i valori "non realistici" ed offrire una maggiore coerenza del dataset.

In aggiunta, è stata effettuata una selezione sulle marche presenti nel dataset. Sono stati esclusi i campioni appartenenti a marche con meno di 500 osservazioni. Le marche poco rappresentate non consentirebbero al modello di apprendere efficacemente degli schemi intrinseci e, peraltro, potrebbero compromettere la generalizzazione del modello.

Feature	Dtype
anno produzione	int64
marca	object
modello	object
allestimento	object
carrozzeria	object
trasmissione	object
condizione	float64
chilometraggio	float64
colorazione	object
colore interni	object
prezzo	int64

Tabella 2: Studio dei domini delle caratteristiche

Verifica dei valori nulli. In Figura 4 è mostrata la quantità di campioni con valori nulli per ciascuna caratteristica nella versione iniziale del dataset. La presenza di dati mancanti causa una visione non del tutto completa da parte del modello. Dunque, occorre stimare i valori dei dati mancanti sulla base di quelli disponibili (**data imputation**) oppure mitigare il problema andando a scartare basi di esperienza (rimozione di campioni o caratteristiche).

Imputazione dei dati. La sfida più impegnativa risiede nell’evitare l’introduzione di bias, un aspetto tutt’altro che banale. Per questo motivo, le tecniche classiche di imputazione – come l’utilizzo della media o della mediana per le variabili numeriche e l’impiego della moda per quelle categoriche – vengono affiancate a valutazioni empiriche specifiche del contesto.

Per le variabili categoriali, la strategia adottata è quella di sostituire i valori mancanti con il valore più frequente (**moda**) all’interno di gruppi specifici di dati. Ad esempio, per ciascun modello di auto, viene calcolata la moda m della *carrozzeria* e della *trasmissione*, sostituendo i valori mancanti con m per quel determinato modello. Analogamente, per variabili come la *colorazione*, *colore interni*, la moda viene calcolata raggruppando per marca.

Per variabili numeriche come la *condizione* e il *chilometraggio*, vengono utilizzate le **medie** in gruppi basati sull’anno di produzione. Questo approccio consente di mantenere coerenza con il dataset, assumendo che i veicoli di età simile abbiano una condizione e un chilometraggio simili.

Verifica dei duplicati. Lo step di verifica dei dati duplicati permette di assicurare l’unicità e la coerenza dei dati, eliminando le ridondanze che possono portare a sovra-rappresentazioni di determinate

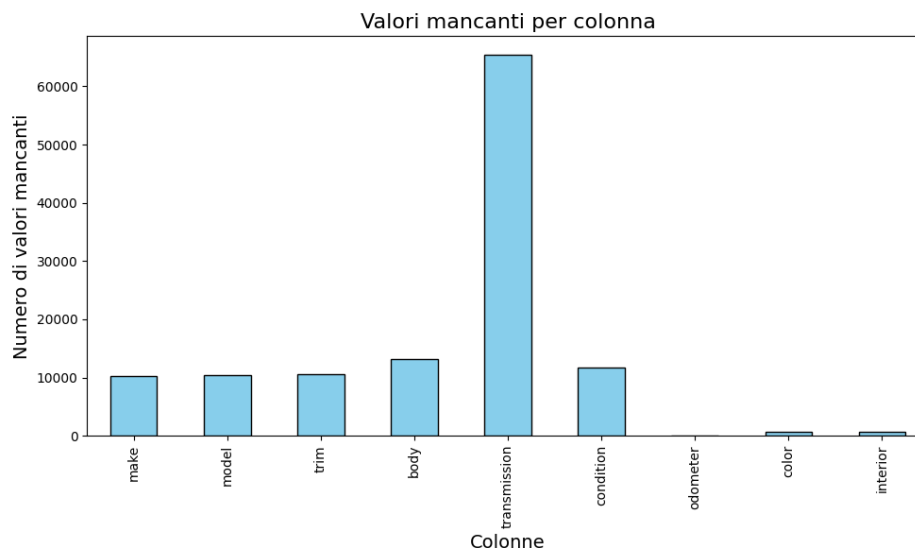


Figura 4: Studio dei valori nulli

osservazioni, inficiando, ineluttabilmente, la genuinità delle predizioni del modello. Fortunatamente, dopo le operazioni svolte in precedenza, il dataset non presenta campioni duplicati.

Conversioni dei tipi. Dalla Tabella 2 si nota che le feature chilometraggio e condizione sono inizialmente rappresentate come *float*. Tuttavia, è stato necessario eseguire conversioni in intero per garantire una maggiore coerenza e semplicità nell'interpretazione delle variabili.

Nella classificazione. Per affrontare il problema con un approccio di classificazione, il target originale, ovvero il prezzo, è stato discretizzato in fasce di prezzo predefinite:

1. 0 – 5000.
2. 5000 – 10000.
3. 10000 – 14000.
4. 14000 – 20000.
5. 20000+.

Questa suddivisione è stata scelta per differenziare adeguatamente le qualità, cercando al contempo di adattarsi alla distribuzione originale del dataset. Un'alternativa comune per la discretizzazione del prezzo sarebbe stata l'uso dei quantili, che garantiscono un bilanciamento tra le classi dividendo il

dataset in gruppi di dimensioni simili. Tuttavia, questa scelta avrebbe potuto portare a categorie meno intuitive per gli utenti finali, con fasce di prezzo non necessariamente coerenti.

2.2.2 Il problema delle scale

Codifica delle caratteristiche categoriche. Dalla Tabella 2 si può notare la presenza di una serie di caratteristiche categoriche. Tuttavia, le variabili categoriali non possono essere direttamente utilizzate in modelli di machine learning senza un adeguato processo di codifica.

Label encoding. La variabile 'trasmissione' ha solo due categorie, che sono mutuamente esclusive: "manuale" e "automatico". Per questo motivo, è possibile trasformare facilmente queste categorie in numeri, assegnando ad esempio 0 a "manuale" e 1 a "automatico".

CatBoost Encoding. Per la gestione delle variabili categoriche ad alta cardinalità, si è scelto di utilizzare il *CatBoost Encoding* per diversi motivi, ma soprattutto per mitigare le criticità riscontrabili con altre tecniche. Il one hot encoding rischia di aumentare notevolmente e inutilmente la dimensionalità del dataset. Il target encoding classico rischia di introdurre overfitting per categorie poco rappresentate.

D'altro canto il CatBoost encoding integra i benefici del target encoding attenuando al contempo i suoi svantaggi. In particolare, introduce un parametro di regolarizzazione α che bilancia in modo controllato l'informazione locale (media del target per categoria) con quella globale (media del target sull'intero dataset). Questa regolarizzazione riduce il rischio di overfitting su categorie rare.

La codifica viene definita dalla seguente formula:

$$encoded_value(c) = \frac{\sum_{i=1}^{N_c} y_i + \alpha \cdot \bar{y}}{N_c + \alpha}$$

dove:

- N_c è il numero di occorrenze della categoria c nel training set;
- $\sum_{i=1}^{N_c} y_i$ rappresenta la somma dei target associati alla categoria c ;
- $\bar{y}$ è la media globale del target calcolata sull'intero dataset;
- α è il parametro di regolarizzazione (con $\alpha \geq 0$) che determina il compromesso tra la stima locale e quella globale.

Il parametro α gioca un ruolo fondamentale:

- Se $N_c \ll \alpha$, l'effetto della regolarizzazione è dominante, e la stima tende verso la media globale $\bar{y}$. Questo è particolarmente utile per categorie rare, per le quali la stima locale sarebbe instabile.
- Se $N_c \gg \alpha$, la stima è prevalentemente influenzata dalla media locale della categoria, sfruttando appieno l'informazione specifica.

Durante la fase di **inference**:

- Le variabili categoriche vengono trasformate utilizzando le statistiche apprese durante il training.
- Per le categorie non presenti nel training set viene assegnato il valore della media globale $\bar{y}$, garantendo coerenza.

In sintesi, l'adozione del CatBoost Encoding permette di combinare i vantaggi del target encoding (cioè l'utilizzo di informazioni rilevanti per il target) con una robusta strategia di regolarizzazione.

Equiparare le scale. Le distribuzioni delle caratteristiche sono tendenzialmente diverse. Attributi con grosse differenze di scala provocano problemi per il machine learner perché potrebbe sottostimare/sovrastimare la rilevanza di una caratteristica. Lo **scaling** è un'operazione che consiste nel modificare i valori delle caratteristiche allo scopo di portarli tutti in un unico dominio. Lo scopo principale è evitare che le scale più ampie possano monopolizzare la fase di apprendimento.

In particolare si è deciso di adottare la ben nota tecnica della "*standardizzazione*" (**z-score normalization**), la quale rende la distribuzione normale con media 0 ($\bar{x}$) e deviazione standard 1 (σ).

$$x' = \frac{x - \bar{x}}{\sigma}$$

Rimozione outlier. Gli outlier, o valori anomali, rappresentano osservazioni che si discostano significativamente dal comportamento generale dei dati. La loro presenza può influenzare negativamente le prestazioni dei modelli di machine learning, introducendo distorsioni nei risultati. Le osservazioni con un valore assoluto di $z - score$ superiore a 3 (*threshold*) sono state considerate outlier e rimosse.

2.2.3 Ingegneria delle caratteristiche

Calcolo dell'età. Invece di utilizzare direttamente l'anno di produzione (ad esempio 1990), si è scelto di calcolare l'età del veicolo come la differenza tra l'anno corrente di riferimento (2015, in questo caso) e l'anno di produzione. Così facendo, la variabile "*anno produzione*" è stata sostituita da "età".

Eliminazione caratteristiche con bassa varianza. Una delle strategie di selezione delle caratteristiche è la rimozione delle feature che presentano una varianza estremamente bassa. La varianza di una caratteristica misura il grado di dispersione dei suoi valori: se una variabile assume valori quasi costanti su tutto il dataset, il suo contributo informativo per il modello è trascurabile.

Matematicamente, la varianza di una variabile X è definita come

$$Var(X) = \frac{1}{N} \cdot \sum_{i=1}^n (X_i - \mu)^2$$

dove N è il numero totale di osservazioni e μ è la media della variabile X .

Per ridurre la dimensionalità del dataset, se la varianza di una feature è inferiore ad una soglia prestabilita (0.1), essa viene rimossa dal dataset.

2.2.4 Bilanciamento delle classi

Dopo tutte le operazioni effettuate per garantire una certa soglia di qualità dei dati, per eseguire il task di classificazione, si è deciso di affrontare la problematica relativa al bilanciamento delle classi. In Figura 5 sono riportate due visualizzazioni rispetto al bilanciamento delle classi.

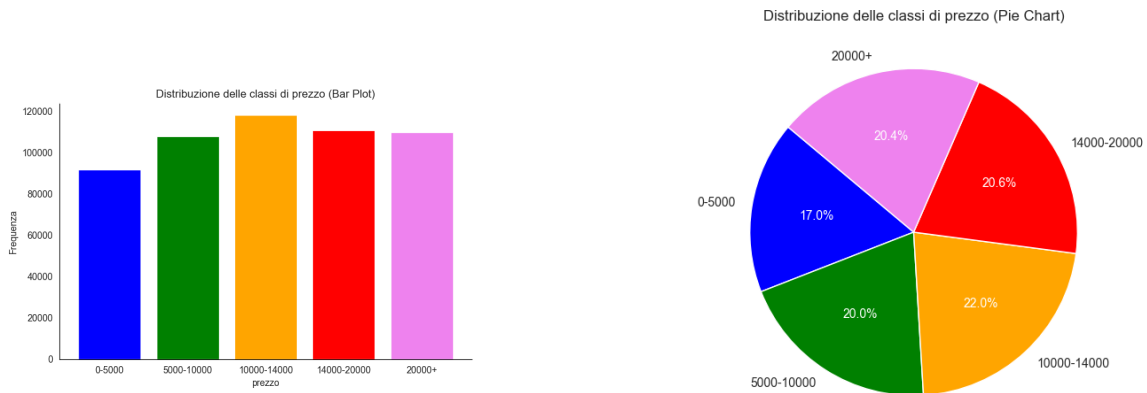


Figura 5: Bilanciamento delle classi

Nel machine learning supervisionato, il bilanciamento delle classi è un elemento fondamentale per garantire che il modello possa generalizzare efficacemente su nuovi dati. Quando una classe è significativamente più rappresentata rispetto alle altre, il modello rischia di imparare a classificare

quasi esclusivamente la classe dominante, mostrando scarsa capacità di riconoscere le classi meno rappresentate.

L'analisi della distribuzione delle classi (Figura 5) ha evidenziato una forte disparità tra classe maggioritaria e classi minoritarie. Per mitigare questo squilibrio, si è adottata la tecnica di **oversampling** con SMOTE (Synthetic Minority Over-sampling Technique). SMOTE genera nuovi campioni sintetici per le classi meno rappresentate, sfruttando la varianza delle caratteristiche esistenti per interpolare dati realistici. Questo approccio permette di ridurre il rischio di overfitting, migliorando la capacità del modello di apprendere pattern significativi anche nelle classi meno frequenti.

3 Sviluppo del modello

3.1 Selezione del modello

La scelta del modello di Machine Learning è una fase cruciale nello sviluppo di un sistema predittivo, soprattutto quando la relazione tra le variabili indipendenti e il target non segue un andamento lineare. In questo contesto, il `RandomForestRegressor` è stato selezionato per la sua capacità di modellare relazioni complesse e non lineari senza dover specificare esplicitamente una funzione matematica.

Perché? Uno dei motivi principali per cui il `RandomForest` è particolarmente adatto a questo problema è la sua flessibilità nella gestione della non linearità nei dati. Molti modelli di regressione tradizionali, come la regressione lineare o polinomiale, richiedono l'assunzione di una forma funzionale specifica tra le variabili indipendenti e il target. Tuttavia, nel caso della valutazione delle automobili, il valore finale è influenzato da un'ampia gamma di fattori (anno di produzione, chilometraggio, marca, modello, condizione del veicolo, ecc.), che interagiscono in modi complessi e difficilmente rappresentabili con una singola funzione matematica. Ad esempio, l'andamento del prezzo di un'auto non segue perfettamente una progressione/recessione rispetto al chilometraggio, in quanto ci sono altri fattori da tener conto (modello, anno di produzione, ecc.).

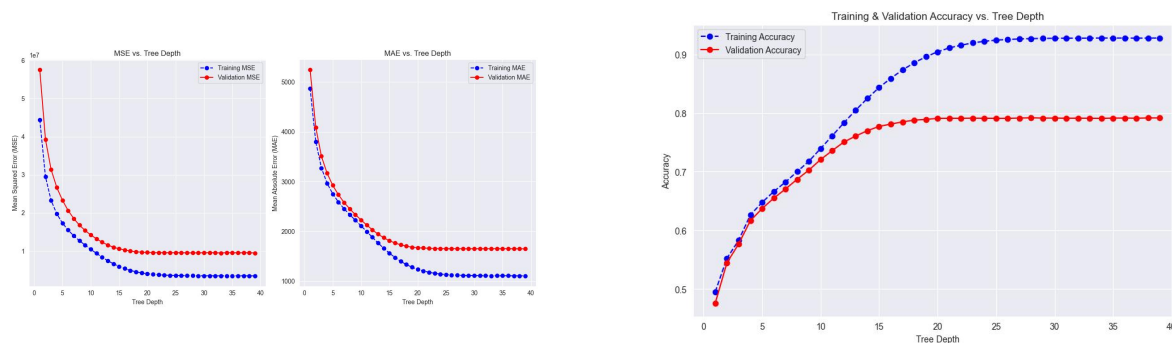


Figura 6: Validation loss e accuracy al variare di `max_depth`

Il `RandomForest`, essendo basato su una collezione di alberi decisionali, è in grado di catturare queste relazioni senza bisogno di assumere una struttura predefinita.

Funzionamento generale. Il `RandomForest` è un algoritmo di apprendimento supervisionato basato su una collezione di alberi decisionali. Si tratta di un metodo di **ensemble** learning, in cui molteplici

alberi vengono addestrati su sottoinsiemi del dataset e le loro predizioni vengono poi aggregate per ottenere un risultato robusto e preciso.

Il modello funziona grossomodo secondo i seguenti passaggi:

1. **campionamento bootstrap.** Vengono generati diversi sottoinsiemi del dataset tramite campionamento con sostituzione, garantendo una maggiore diversità nei dati utilizzati per addestrare ogni albero.
2. **Addestramento.** Ogni albero viene addestrato su un sottoinsieme diverso del dataset e utilizza una selezione casuale delle feature per effettuare gli split. Questo introduce un'ulteriore forma di diversificazione e previene l'overfitting.
3. **Aggregazione.** Il valore finale della previsione è ottenuto combinando i risultati di tutti gli alberi, rendendo il modello più stabile e meno sensibile alla varianza dei singoli alberi. Nel caso di regressione, il valore finale è ottenuto facendo la media delle predizioni di tutti gli alberi, mentre nel caso di classificazione, il risultato viene determinato tramite voto di maggioranza.

3.1.1 Fase di addestramento

Configurazione degli iperparametri. La configurazione degli iperparametri è in generale un aspetto rilevante nell'addestramento di un modello, poiché influisce direttamente sulle sue prestazioni, sulla capacità di generalizzazione e sull'efficienza computazionale. Durante tale scelta, è stato necessario trovare un compromesso tra accuratezza, complessità computazionale e rischio di overfitting.

Per la regressione. Il parametro *n_estimators* controlla il numero di alberi: un numero eccessivamente elevato generalmente migliora la stabilità, ma aumenta inesorabilmente il tempo di addestramento e il consumo di memoria. *n_estimators* = 300 alberi rappresentano un buon compromesso tra accuratezza e efficienza computazionale.

Il parametro *max_features* determina quante feature vengono considerate a ogni split dell'albero. *max_features* = *sqrt(n_features)* introduce maggiore diversità tra gli alberi, riducendo il rischio di overfitting e migliorando la generalizzazione.

Il parametro *min_samples_split* definisce il numero minimo di campioni richiesto per suddividere un nodo. Valori molto bassi possono portare a suddivisioni troppo frequenti, causando overfitting e modelli poco generali. *min_samples_split* = 10 è un valore adeguato per garantire una sufficiente granularità nelle suddivisioni, senza eccessivo rumore.

Il parametro *random_state* controlla la riproducibilità dei risultati fissando il seme del generatore di numeri casuali. *random_state* = 20 garantisce la consistenza dei risultati tra diversi esperimenti.

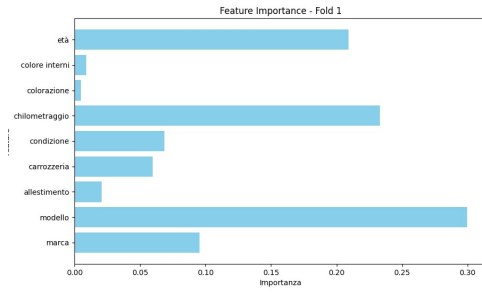
Il parametro *bootstrap* definisce se il campionamento dei dati per ciascun albero avviene con sostituzione o meno. *Bootstrap* = *True* significa che gli alberi vengono addestrati su sottoinsiemi casuali con rimpiazzo, generando diversità tra gli alberi e migliorando la capacità del modello di generalizzare.

Il parametro *criterion* indica il criterio che il modello utilizza per selezionare il miglior punto di split durante la costruzione degli alberi. Tra i possibili valori, abbiamo scelto *squared_error*, che cerca di minimizzare la somma degli errori quadratici (MSE) in ogni nodo. Questa scelta è stata fatta per la sua capacità di ridurre la varianza tra i dati nei nodi terminali, contribuendo così a limitare il rischio di overfitting.

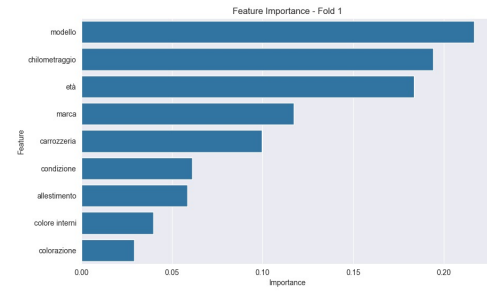
Il parametro *max_depth* definisce la profondità massima di un singolo albero decisionale. Nella scelta del valore di *max_depth*, è importante considerare alcuni aspetti: all'aumentare di questo valore, il modello impara di più dai dati e fornisce previsioni più accurate, ma aumenta anche il rischio di overfitting. Per determinare il valore *ideale*, si è tracciato un grafico dell'andamento della validation loss (in base a MAE e MSE) all'aumentare del valore di *max_depth* (Figura 6). Si è riscontrato che *max_depth* = 18 (nel caso della regressione) fosse il valore ottimale: valori più elevati avrebbero comportato uno spreco di risorse (memoria e tempo) e un rischio maggiore di overfitting.

Il parametro *min_samples_leaf* definisce il numero minimo di campioni che devono essere presenti nei nodi figli di un determinato nodo prima che si possa effettuare uno split. Se, dopo uno split, i nodi figli avessero meno di *min_samples_leaf* campioni, il nodo padre non verrebbe splittato. Più basso è il valore di *min_samples_leaf*, maggiore sarà la profondità degli alberi, il che può migliorare le prestazioni del modello, permettendo a ogni albero di adattarsi meglio ai dati. Tuttavia, un valore troppo basso può portare a un overfitting maggiore, poiché il modello impara a "memorizzare" troppo bene i dettagli specifici del dataset di addestramento. Considerata la varianza abbastanza elevata del dataset, si è scelto di impostare *min_samples_leaf* = 4 per evitare un eccessivo approfondimento degli alberi e il rischio di overfitting.

Per la classificazione. Per il problema di classificazione, la maggior parte degli iperparametri scelti è stata mantenuta coerente con quelli adottati per la regressione, al fine di garantire una base di confronto solida tra i due approcci. Tuttavia, alcune differenze chiave sono state introdotte per adattare il modello alle peculiarità della classificazione:



(a) Regressione



(b) Classificazione

Figura 7: Importanza delle caratteristiche

- criterio di suddivisione (*criterion*). Mentre nella regressione è stato utilizzato *squared_error*, per la classificazione si è optato per *gini*.
- Profondità massima degli alberi (*max_depth* = 20). Mentre nella regressione il valore ottimale risultava 18, per la classificazione è stato osservato un comportamento diverso. Le analisi relative alla validation accuracy (plottata in Figura 6) ha mostrato che valori maggiori di *max_depth* = 20 non portassero vantaggi significativi.

3.2 Tecniche di valutazione

La valutazione di un modello di Machine Learning è una fase cruciale per garantirne l'affidabilità e la capacità di generalizzazione. Un errore comune è valutare il modello su un unico **train-test split**, il che può portare a risultati distorti a causa della specifica suddivisione scelta. Per mitigare questo problema, esistono diverse tecniche di validazione, tra cui la **k-fold cross validation**, che consente di ottenere una stima più robusta delle prestazioni del modello. La validazione incrociata è un metodo statistico che consiste nella ripetuta partizione e valutazione dell'insieme dei dati di partenza. Ogni elemento del dataset è assegnato ad un unico gruppo durante l'intera procedura di validazione, altrimenti il rischio è di mischiare dati di training con dati di test, andando a ricadere in un caso di **data leakage**. Mediante questa tecnica si fanno k validazioni del modello allo scopo di considerare ogni *fold* una volta sola come test set. Ad ogni iterazione vengono applicate le operazioni di **preprocessing** sui dati solamente sul training set.

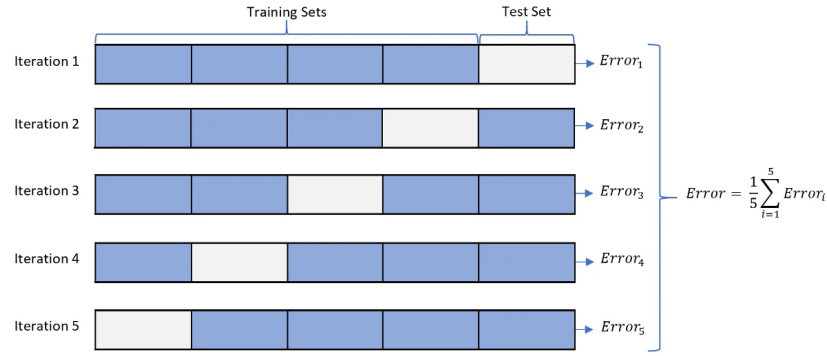


Figura 8: Validazione incrociata

3.2.1 Metriche di validazione

Per la classificazione. Sono state utilizzate alcune delle metriche standard che forniscono indicazioni circa le prestazioni ottenute. Osservando la Figura 9 sono stati calcolati:

$$precision = \frac{TP}{TP + FP} \quad recall = \frac{TP}{TP + FN}$$

$$f1 - score = \frac{2 \cdot precision \cdot recall}{precision + recall} \quad accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

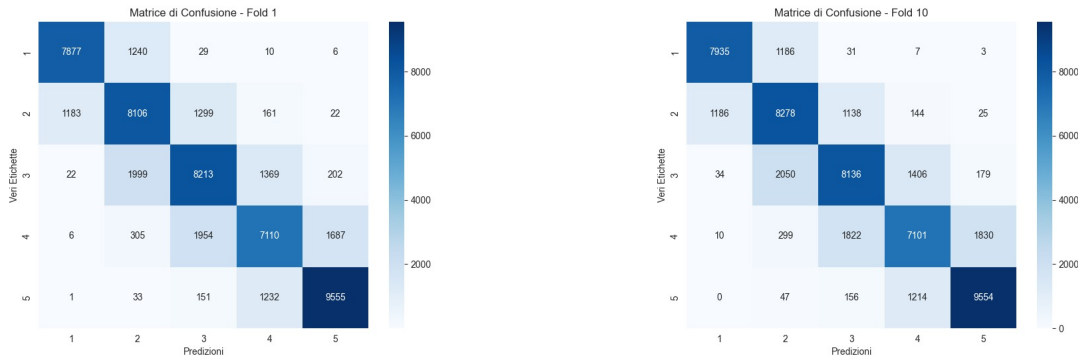


Figura 9: Matrice delle confusioni (primo e ultimo fold)

In Tabella 3 sono visualizzati i risultati ottenuti fold per fold.

Per la regressione. Sono state utilizzate delle metriche standard allo scopo di comprendere l'adattabilità del modello finale rispetto al dataset.

fold	accuracy	precision	recall	f1-score
1	0.7599	0.7594	0.7599	0.7589
2	0.7614	0.7605	0.7614	0.7600
3	0.7604	0.7595	0.7604	0.7592
4	0.7612	0.7605	0.7612	0.7601
5	0.7611	0.7606	0.7611	0.7602
6	0.7629	0.7623	0.7629	0.7618
7	0.7609	0.7602	0.7609	0.7597
8	0.7647	0.7640	0.7647	0.7636
9	0.7613	0.7607	0.7613	0.7602
10	0.7626	0.7618	0.7626	0.7612

Tabella 3: Valori metriche di validazione (classificazione)

$$MAE = \frac{\sum_{i=1}^n |\hat{y} - y|}{n} \quad MSE = \frac{\sum_{i=1}^n (\hat{y} - y)^2}{n}$$

$$RMSE = \sqrt{MSE} \quad MAPE = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100$$

In Tabella 4 sono visualizzati i risultati ottenuti fold per fold.

fold	MAE	MSE	RMSE	MAPE
1	1523.729	7924227.413	2815.000	0.167
2	1550.911	8283304.946	2878.073	0.167
3	1525.650	8973708.403	2995.614	0.168
4	1530.215	7879336.225	2807.015	0.167
5	1539.077	8816325.526	2969.229	0.165
6	1549.617	8735598.148	2955.604	0.165
7	1557.969	8581746.010	2929.461	0.168
8	1544.617	8156220.896	2855.909	0.167
9	1527.131	7820900.663	2796.587	0.170
10	1531.875	8398450.020	2898.007	0.166

Tabella 4: Valori metriche di validazione (regressione)

Dai risultati ottenuti e visualizzati, è possibile notare come le prestazioni del modello (sia per la classificazione che per la regressione) non variano considerevolmente tra una partizione e l'altra, indicando tutto sommato una buona capacità di generalizzazione sui dati.

4 Usabilità

L'usabilità di un sistema software, e in particolare di un modello di intelligenza artificiale, è un aspetto cruciale che ne determina il valore pratico. Un modello accurato, ma difficile da integrare, rischia di perdere gran parte del suo potenziale impatto.

4.1 Leggi di Lehman

Nel campo dell'ingegneria del software, le leggi di Lehman forniscono un quadro teorico utile per comprendere come i sistemi software si evolvono nel tempo. In linea di principio, occorre capire come mantenere nel tempo lo stato del modello. Un sistema software che non cambia nel tempo è inutile.

Due di queste leggi sono particolarmente rilevanti per il rilascio e l'usabilità di un modello di machine learning:

- **cambiamento continuo:** un sistema software non può rimanere statico, e per rimanere utile deve adattarsi ai cambiamenti dei requisiti, delle tecnologie e dell'ambiente operativo. Nel caso di un modello di machine learning, questo significa che deve essere aggiornato con nuovi dati e tecniche per mantenere la sua accuratezza e rilevanza.
- **Complessità crescente:** man mano che un sistema viene modificato ed esteso, tende a diventare più complesso, a meno che non vengano adottate strategie specifiche per gestire tale crescita. Questo principio evidenzia l'importanza di un'architettura modulare e di un'interfaccia ben progettata, in modo da mantenere il modello facilmente utilizzabile e aggiornabile.

4.2 Interfaccia grafica

Per facilitare l'interazione con il modello addestrato ed, in generale, rendere, per quanto possibile, migliore l'interazione tra l'utente finale e la "macchina" è stata sviluppata un'interfaccia grafica utilizzando Streamlit, un framework open-source che permette di creare applicazioni web interattive in modo rapido e semplice.

L'interfaccia realizzata consente agli utenti di:

1. caricare i dati del veicolo.
2. Visualizzare il risultato in modo chiaro con un'indicazione del valore stimato.

Stima del Prezzo di un'Auto

Inserisci la marca:

Inserisci il modello:

Inserisci l'anno di produzione:

Inserisci l'allestimento:

Inserisci la carrozzeria:

Inserisci la condizione dell'auto:

Inserisci la condizione dell'auto:

Inserisci il chilometraggio:

Inserisci il colore esterno:

Inserisci il colore degli interni:

Il prezzo stimato è: 12107.57€

Figura 10: Interfaccia grafica

Piccolo ma importantissimo DISCLAIMER. È fondamentale evidenziare che le valutazioni fornite non rappresentano una quotazione ufficiale o definitiva del valore di un'auto. Il modello è stato addestrato su un dataset specifico e può essere indubbiamente soggetto a errori o limitazioni. L'output deve essere interpretato come un supporto alla decisione e non come un valore assoluto o vincolante.

4.2.1 Perché Streamlit?

Streamlit è stato scelto per diversi motivi:

- **semplicità** d'uso e **velocità** di sviluppo. Permette di costruire interfacce grafiche con poche righe di codice, senza la necessità di sviluppare una web app complessa; inoltre, essendo pensato per applicazioni di data science, si integra perfettamente con modelli di machine learning.
- **Interattività** immediata. Offre componenti predefiniti per il caricamento di immagini e l'interazione con i risultati del modello.

5 Considerazioni finali

Il progetto Carval ha dimostrato come l'intelligenza artificiale possa essere uno strumento potente per la valutazione automatizzata del valore delle automobili. Attraverso l'analisi di un ampio dataset, è stato possibile sviluppare un sistema capace di offrire stime accurate e replicabili.

Tuttavia, come ogni sistema data-driven, Carval presenta alcune limitazioni e margini di miglioramento, che possono essere affrontati in sviluppi futuri per aumentarne l'efficienza e l'affidabilità.

5.1 Possibili miglioramenti

Per migliorare le prestazioni e l'affidabilità di Carval, si possono considerare diversi interventi, sia a livello di dati che di modello predittivo.

5.1.1 Nei dati

Espansione. L'integrazione di dati provenienti da più mercati permetterebbe di rendere il modello più generale e adattabile a contesti diversi. L'aggiornamento con dati più recenti migliorerebbe la capacità del modello di adattarsi alle fluttuazioni del mercato.

Granularità. L'inclusione di informazioni più dettagliate, come la cronologia delle manutenzioni, gli incidenti subiti o il numero di proprietari precedenti, potrebbe affinare ulteriormente le previsioni.

5.1.2 Nel modello

Sperimentazione. A causa delle tempistiche di realizzazione e consegna del progetto, non è stato possibile dedicare tempo sufficiente alla sperimentazione di diversi modelli di Machine Learning. Sebbene il RandomForestRegressor sia stato scelto per la sua robustezza e capacità di gestire dati non lineari, sarebbe stato interessante valutare altre alternative.

Ottimizzazione. L'uso di strategie avanzate di configurazione degli iperparametri, magari tramite algoritmi genetici, potrebbe migliorare l'accuratezza generale del modello.